Computational algebra

RadicalRoots

Exact radicals from algebraic roots

Structural reductions, constructive Galois theory,
and branch-aware reference implementations

Prepared for Vladimir Reshetnikov
8 September 2026
Reference implementation 0.1.0

Abstract

An algebraic `Root` object specifies both a polynomial equation and a particular complex root. Producing radicals therefore requires more than finding a solvable equation: every radical branch must be made consistent with that embedding. This article develops a two-layer solution. A fast layer recognizes polynomial composition, powers, generalized reciprocal symmetry and Dickson polynomials. A general layer constructs an embedded splitting field, adjoins a carefully chosen cyclotomic field, computes actual field automorphisms and applies finite Fourier resolvents along a prime-index normal series. It produces explicit principal-power expressions with exactly selected root-of-unity corrections.

The unbounded mathematical construction covers every algebraic number over $\mathbb{Q}$ that is expressible by radicals. The accompanying software implements that construction using computer-algebra primitives; it is a reference implementation, not an unconditional performance or backend-completeness claim. The Python implementation has executable regression results. The Wolfram Language implementation and its native tests are supplied but were not executed because the connected Wolfram service returned HTTP 404.

Deliverables: Wolfram Language package; Python/SymPy implementation; regression tests and recorded results; source-notebook inspection notes; this article in editable LaTeX and compiled PDF.

Contents

1 Problem, contract, and scope **2**

2 What the notebook contributes **2**

3 The mathematical existence criterion **3**

4 Fast structural reductions **3**
4.1 Low degree, powers, and composition . . . 4
4.2 Dickson polynomials and coherent inverse pairs . . . 4
4.3 Generalized reciprocal symmetry . . . 5
4.4 Explicit extension hints . . . 5

5 The two motivating examples **5**
5.1 The sextic: a cubic hidden by $X - 1/X$ . . . 5
5.2 The quintic: an inverse Dickson value . . . 6

6 A complete constructive route through a splitting field **6**
6.1 Build the normal closure with its embedding . . . 6
6.2 Adjoin roots of unity before taking resolvents . . . 7
6.3 Recover actual automorphisms from a primitive element . . . 7

7 Constructing a prime cyclic normal series **8**

8 Fourier descent, zero resolvents, and exact phases **8**
8.1 The local identity . . . 8
8.2 Principal-root selection . . . 9
8.3 The bottom field . . . 9
8.4 Pseudocode . . . 9

9 Correctness and completeness of the construction **10**
9.1 The supplied-model caveat . . . 10

10 Certification details and software architecture **11**
10.1 Keep field values and radical syntax separate . . . 11
10.2 Exactness of the chosen embedding in Python . . . 11
10.3 Failure taxonomy . . . 12

11 Complexity: finite does not mean small **12**

12 Using the supplied implementations **13**
12.1 Wolfram Language . . . 13
12.2 Python and SymPy . . . 13
12.3 Supplying a known embedded field . . . 14

13 Validation record and limitations **14**

14 Related work and practical next steps **15**

A Package map and reproducibility **16**

1 Problem, contract, and scope

The motivating question asks why `ToRadicals` does not convert certain explicit algebraic numbers despite known radical formulas [1]. Wolfram's documentation explicitly notes that some higher-degree solvable polynomials are not converted by that function [2]. Consequently, an unchanged `Root` is not a proof of nonsolvability.

The input considered here is an exact algebraic number $\alpha \in \overline{\mathbb{Q}}$ with its specified embedding in $\mathbb{C}$. The ordinary Wolfram example is `Root[f&,k]` for a rational-coefficient polynomial. The Python front end accepts a rational polynomial and a zero-based SymPy root index. It first selects the actual root and then works with its *minimal polynomial*. This distinction is important for a reducible input: a rational root of a polynomial having an additional nonsolvable factor is still rational.

Definition 1.1 (Output language)**.** A radical expression is built from rational constants and i, by addition, multiplication and rational powers, with nonzero denominators. Powers use principal complex values. Equivalently, division is a negative integer power. No `Root`, `CRootOf`, `AlgebraicNumber`, symbolic parameter, inexact floating-point constant, trigonometric function or exponential function is permitted in a successful output.

The syntax is deliberately stricter than "a short expression involving algebraic special-function values." For instance, $\cos(\pi/7)$ is algebraic and solvable, but is not an output of the specified grammar. Conversely,

$$\zeta_m = (-1)^{2/m}$$

is a permitted radical expression: with the principal logarithm of -1, its value is $e^{2\pi i/m}$. The exponential is explanatory notation, not an output head. The cases $m = 1, 2$ give $1, -1$.

A successful public result must satisfy both the grammar test and the selected-root equality test. Wolfram uses `RootReduce[e-alpha]===0`; the Python implementation compares exact minimal polynomials and then performs a separation-bound root-identity test. These are CAS-level checks, not proof-assistant certificates. Internal `Root` and number-field objects are allowed while constructing and verifying the answer; they must not remain in the returned expression.

The package does not solve parameter-dependent equations, decide whether an arbitrary special-function expression is algebraic, minimize radical depth, or guarantee the shortest printed expression. These are different problems. The default finite resource limits also mean that a solvable input can yield a resource-limit report rather than radicals.

2 What the notebook contributes

The supplied `FindExtension.nb` is an extensive exploratory session. Inspection of the archive, without evaluating it, located 220 Input cells. The accompanying diagnostic transcription records those cells and the original notebook line numbers. It is not presented as an executable export: some special boxes and quantifier notation remain.

The essential strategy of `findExtension` is to divide the minimal polynomial by a prospective quadratic X^2+pX+q, eliminate the condition that the remainder vanish, and select a comparatively simple algebraic value of a coefficient. The cubic variant `findExtension3` applies the same idea to $X^3 + pX^2 + qX + r$. This is a useful way of looking for an extension over which the original polynomial has a small factor. The subsequent steps factor over that extension and select the factor containing the desired root.

Additional experiments project roots onto real or imaginary parts, search subset sums of conjugates, recognize candidate minimal polynomials from high-precision values, and use manually selected radical or cyclotomic extensions. Those ideas motivate the fast layer and the extension-hint interface. They do not, by themselves, constitute an exhaustive algorithm. A solvable group need not expose the desired tower through quadratic or cubic factors alone.

Several engineering changes are required to turn this exploration into a reusable solver. The notebook includes `While[True]` searches, selection of the first candidate without a general empty-result contract, references to private simplification symbols and stateful constructs such as previous outputs. Its initial complexity function counts `Sin` and `Cos` in a depth measure; a small cost therefore does not imply a radical-only answer. In two early cells the literal private name `Simplify‘SimpifyCount` differs from the later `Simplify‘SimplifyCount`. The replacement relies on neither spelling.

Numerical recognition is useful for proposing an algebraic identity, but a 400-digit match is not the same as establishing that identity. Similarly, `PossibleZeroQ` is not the explicit branch certificate chosen here. The replacement separates proposal generation, exact root selection, radical-only syntax, resource limits and mathematical nonsolvability. The source-review directory contains the detailed mapping and archive hashes.

3 The mathematical existence criterion

Let $f \in \mathbb{Q}[X]$ be the monic irreducible minimal polynomial of α, let $L \subset \mathbb{C}$ be its splitting field, and put $G = \mathrm{Gal}(L/\mathbb{Q})$. Galois's solvability theorem states that a characteristic-zero polynomial is solvable by radicals precisely when its Galois group is solvable [4, Theorem 5.34].

Proposition 3.1 (One specified root is enough)**.** *The algebraic number α is expressible by radicals over $\mathbb{Q}$ if and only if G is solvable.*

Proof. If the whole polynomial is solvable by radicals, so is its selected root. Conversely, suppose α belongs to a radical tower. Each embedding of $\mathbb{Q}(\alpha)$ into an algebraic closure extends to the algebraic fields containing that tower. The images of its radical generators remain roots of the corresponding power equations. Thus every conjugate of α also belongs to a radical extension. Taking the compositum of the finitely many such towers still gives a radical extension containing every root of f. Apply the polynomial solvability theorem. □

This statement requires the *minimal* polynomial. It does not say that $\mathbb{Q}(\alpha)/\mathbb{Q}$ is normal, nor that the necessary radical tower must be contained in $\mathbb{Q}(\alpha)$. For example, a cubic may require auxiliary complex radicals even when the selected root is real. Searching only inside the degree-deg f root field is therefore not a complete approach.

For a finite group, solvability can be decided by repeatedly taking commutator subgroups. The sequence either reaches the identity or stabilizes at a nontrivial perfect subgroup. The constructive algorithm below refines this into a normal series with prime cyclic quotients; it needs the actual field action, not merely the abstract isomorphism type of the group.

4 Fast structural reductions

The fast layer operates on equations, generates candidates, and then checks which candidate equals the selected input. It does not infer a branch from a numerical ordering alone. Affine depression is

performed first when needed: for a monic degree-n polynomial with next coefficient a_{n-1}, substitute $X = Y - a_{n-1}/n$.

4.1 Low degree, powers, and composition

Degrees one and two use the standard formulas. For a depressed cubic $Y^3 + PY + Q = 0$, set

$$\Delta = \frac{Q^2}{4} + \frac{P^3}{27}, \qquad t = -\frac{Q}{2} + \sqrt{\Delta}, \qquad u = t^{1/3}.$$

Choose the other sign of $\sqrt{\Delta}$ when the first t is zero, unless $P = Q = 0$, which is handled separately. The complete coherent candidate list is

$$Y_k = \zeta_3^k u - \frac{P}{3\zeta_3^k u}, \qquad k = 0, 1, 2.$$

Indeed, the two summands have product $-P/3$, and their cubes sum to $-Q$. This product constraint is what independently chosen cube roots can violate. Quartic candidates come from the CAS's explicit quartic solver and are subsequently checked.

If every nonconstant exponent is divisible by $r > 1$, then $f(X) = g(X^r)$. Solve $g(Y) = 0$ recursively and generate $X = \zeta_r^k Y^{1/r}$, for all $0 \le k < r$. The zero case is harmless. Likewise, a rational polynomial decomposition

$$f = f_1 \circ f_2 \circ \cdots \circ f_s$$

can be solved from the outside inward. Intermediate coefficients may then be radicals. The package does not claim to discover every decomposition over every algebraic coefficient field: rational decomposition is a fast path, not the completeness mechanism.

4.2 Dickson polynomials and coherent inverse pairs

Define

$$D_0(X, a) = 2, \qquad D_1(X, a) = X, \qquad D_n(X, a) = XD_{n-1}(X, a) - aD_{n-2}(X, a).$$

Direct induction gives the key identity

$$D_n(u + a/u, a) = u^n + (a/u)^n. \tag{1}$$

For $a \neq 0$, an equation $D_n(X, a) + c = 0$ is therefore solved by

$$\begin{aligned} t &= \frac{-c + \sqrt{c^2 - 4a^n}}{2}, \qquad u = t^{1/n}, \\ X_k &= \zeta_n^k u + \frac{a}{\zeta_n^k u}, \qquad 0 \le k < n. \end{aligned} \tag{2}$$

Here t satisfies $t^2 + ct + a^n = 0$, so the two n-th powers in (1) sum to $-c$. Because $a \neq 0$, neither quadratic root is zero. The code nevertheless includes a zero guard. The case $a = 0$ reduces to a binomial.

The identity is not a heuristic recognition rule. After depression, the coefficient of X^{n-2} suggests $a = -[X^{n-2}]f/n$; the implementation then checks the *entire* polynomial identity $f - D_n = c$. The same pair $u, a/u$ is used throughout. Taking two unrelated principal n-th roots need not preserve their required product a.

4.3 Generalized reciprocal symmetry

Let $f(X) = \sum_{k=0}^{2m} c_k X^k$ be monic. Suppose $a \in \mathbb{Q}^\times$ satisfies

$$c_{m-j} = a^j c_{m+j} \quad (1 \le j \le m). \tag{3}$$

Necessarily $c_0 = a^m$. Grouping the positive and negative powers yields

$$\frac{f(X)}{X^m} = c_m + \sum_{j=1}^{m} c_{m+j} \left(X^j + (a/X)^j \right).$$

By (1), this equals $g(X + a/X)$, where

$$g(Y) = c_m + \sum_{j=1}^{m} c_{m+j} D_j(Y, a).$$

Thus a degree-$2m$ problem reduces to a degree-m equation and quadratics:

$$X = \frac{Y \pm \sqrt{Y^2 - 4a}}{2}. \tag{4}$$

The implementation enumerates the rational solutions of $a^m = c_0$, then checks every condition in (3). This includes ordinary reciprocity $a = 1$ and anti-reciprocal substitutions $a = -1$.

4.4 Explicit extension hints

The Wolfram interface also accepts a list of already explicit radical constants as `"ExtensionHints"`. It factors over their compositum, selects smaller-degree factors by exact vanishing at the original root, and passes them to the fast solver. This retains the useful part of the notebook workflow without confusing a guessed extension with a proof of success. The Python front end does not currently implement this hint interface; its supplied-field interface serves a different, lower-level purpose.

5 The two motivating examples

5.1 The sextic: a cubic hidden by $X - 1/X$

Consider

$$f(X) = X^6 + X^4 - X^3 - X^2 - 1.$$

The exact identity

$$f(X) = X^3 \left((X - X^{-1})^3 + 4(X - X^{-1}) - 1 \right) \tag{5}$$

exhibits generalized reciprocity with $a = -1$. Put

$$t = \left(\frac{1}{2} + \frac{\sqrt{849}}{18} \right)^{1/3}, \qquad y = t - \frac{4}{3t}, \qquad \alpha = \frac{y + \sqrt{y^2 + 4}}{2}. \tag{6}$$

All roots in this displayed formula are positive real principal roots. The cubic is $y^3 + 4y - 1 = 0$: its Cardano discriminant is $1/4 + 64/27 = 283/108$, whose square root is $\sqrt{849}/18$.

The cubic derivative $3y^2 + 4$ is positive on $\mathbb{R}$, so it has exactly one real root. For that root, $X^2 - yX - 1 = 0$ has one negative and one positive solution. Nonreal y cannot come from a real nonzero X. Hence f has exactly two real roots, and α is the positive one: the requested Wolfram `Root[-1-#^2-#^3+#^4+#^6&,2]`. This is a structural explanation of the conversion, not a lucky extension search. The three cubic candidates, each lifted by both signs in (4), yield all six conjugates.

5.2 The quintic: an inverse Dickson value

The second polynomial is

$$5X^5 - 25X^3 + 25X + 6 = 5\left(D_5(X, 1) + \frac{6}{5}\right).$$

Set

$$u = \left(\frac{-3 + 4i}{5}\right)^{1/5}, \qquad \beta = u + \frac{1}{u}. \tag{7}$$

Here u is the principal fifth root. Its fifth power has modulus one, so u also has modulus one. The inverse-pair identity immediately gives $D_5(\beta, 1) = -6/5$.

To identify the selected branch analytically, write $(-3 + 4i)/5 = e^{i\theta}$ with $\pi/2 < \theta < \pi$. Then the five candidates are $2\cos((\theta + 2\pi k)/5)$, $0 \le k < 5$, and are distinct and real. The angle $\theta/5$ is closest to a multiple of 2π, so $k = 0$ gives the largest root. Thus (7) equals the requested Wolfram root with index 5. The trigonometric notation is used only in this proof of ordering; it is absent from the returned radicals.

The Python suite tests every conjugate of both polynomials, not just these two branches. The sextic's positive real root is Python index 1, and the quintic's largest real root is index 4. These particular correspondences must not be generalized to arbitrary complex-root indices across systems.

6 A complete constructive route through a splitting field

The remaining construction uses only exact algebraic-number operations and finite group computations. Its purpose is completeness in principle, not competition with specialized radical formulas on their natural inputs.

6.1 Build the normal closure with its embedding

Obtain every exact root of the irreducible polynomial f, and construct

$$L = \mathbb{Q}(\alpha_1, \ldots, \alpha_n) \subset \mathbb{C}.$$

A primitive element θ_L represents this field as a rational polynomial quotient. Because all roots of f are included, the field is normal; characteristic zero gives separability. Its degree $d = [L : \mathbb{Q}]$ is the order of its Galois group.

The Wolfram implementation uses `ToNumberField[values,All]`: the `All` is essential because that form is documented to use the smallest common field extension, whereas the automatic form need not [3]. The Python implementation performs successive exact primitive-element adjunctions. For degrees at most four, it may use explicit radical roots *internally* as field generators to avoid a costly `CRootOf` adjunction. That optimization does not replace Fourier descent when the general method is requested.

6.2 Adjoin roots of unity before taking resolvents

Define the squarefree integer

$$m = \mathrm{rad}(d) = \prod_{p|d} p, \qquad C = \mathbb{Q}(\zeta_m), \qquad M = LC. \tag{8}$$

Use $m = 1$ if $d = 1$. Let

$$H = \mathrm{Gal}(M/C).$$

Every prime divisor of $|H|$ divides d, hence divides m. Consequently C contains every primitive prime-order root of unity needed for a prime-quotient normal series of H.

Lemma 6.1 (The cyclotomic base does not hide nonsolvability)**.** *The group H is solvable if and only if $G = \mathrm{Gal}(L/\mathbb{Q})$ is solvable.*

Proof. Restriction identifies H with $\mathrm{Gal}(L/(L \cap C))$. The intersection $L \cap C$ is Galois over $\mathbb{Q}$, and its Galois group is abelian because it is a subquotient of the cyclotomic Galois group. Thus H identifies with a normal subgroup of G with abelian quotient. Subgroups of solvable groups are solvable; conversely, an extension of a solvable group by an abelian group is solvable. □

A common error is to compute automorphisms of $L/\mathbb{Q}$, adjoin a formal ζ_p, and then assume the old automorphisms fix it. Intersections with the cyclotomic field make that assumption invalid. We construct the actual compositum and explicitly retain only automorphisms fixing its chosen ζ_m. No linear-disjointness assumption is made.

6.3 Recover actual automorphisms from a primitive element

Let $M = \mathbb{Q}(\theta)$, with degree D and minimal polynomial F. Every element has a unique coordinate representation

$$a = A(\theta), \qquad A \in \mathbb{Q}[T], \quad \deg A < D.$$

Since $M/\mathbb{Q}$ is Galois, F has D distinct roots θ_j in M. Each gives an automorphism

$$\sigma_j(A(\theta)) = A(\theta_j). \tag{9}$$

Composition is determined by the image of θ: find the unique index k for which $\sigma_i(\theta_j) = \theta_k$. This yields a multiplication table on actual embedded field maps. Select

$$H = \{\sigma_j : \sigma_j(\zeta_m) = \zeta_m\}.$$

The dimension check

$$|H|\varphi(m) = D \tag{10}$$

is an inexpensive but important invariant.

Wolfram enumerates the conjugates of θ as exact roots and obtains coordinates using `ToNumberField`. Python factors F over M; exactly D distinct linear factors certify normality and provide all images. Its field elements are rational coefficient arrays reduced modulo F, and (9) is evaluated by Horner's rule.

An abstract permutation group returned by a Galois-group routine is not silently assigned to a numerically ordered root list. The optional SymPy small-degree group computation is used only to decide solvability, never to attach an unverified action to root labels.

7 Constructing a prime cyclic normal series

We need

$$H = H_0 \triangleright H_1 \triangleright \cdots \triangleright H_r = 1, \qquad H_i \triangleleft H_{i-1}, \quad H_{i-1}/H_i \simeq C_{p_i}, \tag{11}$$

where each p_i is prime. This can be found without enumerating the entire subgroup lattice.

For a current nontrivial subgroup K, generate its derived subgroup $N = [K, K]$. If $N = K$, this nontrivial perfect subgroup proves that the original group is not solvable. Otherwise enlarge N greedily by adjoining an element of K whenever the generated subgroup remains proper. Every subgroup encountered contains $[K, K]$, so its quotient image in $K/[K, K]$ is a subgroup of an abelian group; it is therefore normal in K. When no further proper enlargement is possible, N is maximal proper and normal. The quotient K/N is a simple finite abelian group, hence cyclic of prime order. Repeat within N.

A single pass over the elements of K suffices for the greedy enlargement. Once adjoining an element would generate all of K, making N larger cannot make that same adjunction proper later. The implementation independently checks normality, primality of the index, and that the chosen coset generator σ satisfies $\sigma^p \in N$.

Under Galois correspondence, (11) gives

$$C = M^{H_0} \subset M^{H_1} \subset \cdots \subset M^{H_r} = M,$$

with cyclic prime-degree steps. The next section translates these steps into explicit radicals without choosing a potentially vanishing special resolvent as a generator.

8 Fourier descent, zero resolvents, and exact phases

8.1 The local identity

Fix one step $N \triangleleft K$ with K/N cyclic of prime order p. Choose a representative σ generating the quotient. Let $a \in M^N$, and let $\zeta = \zeta_p \in C$. Define

$$R_j = \sum_{k=0}^{p-1} \zeta^{-jk} \sigma^k(a), \qquad 0 \le j < p. \tag{12}$$

Lemma 8.1 (Fourier descent)**.** *Each R_j lies in M^N, $R_0 \in M^K$, and $R_j^p \in M^K$. Moreover,*

$$\sigma(R_j) = \zeta^j R_j, \qquad a = \frac{1}{p} \sum_{j=0}^{p-1} R_j. \tag{13}$$

Proof. Normality of N shows that every $\sigma^k(a)$ is fixed by N, and N fixes ζ. Thus the resolvents are fixed by N. The relation $\sigma^p \in N$ gives $\sigma^p(a) = a$. Shifting the summation index in (12) now gives the displayed eigenvalue equation. It makes R_0 and R_j^p fixed by σ, hence by $K = \langle N, \sigma \rangle$. Finally, $\sum_{j=0}^{p-1} \zeta^{-jk}$ is p for $k = 0$ and zero otherwise. Summing (12) over j proves the inverse identity. □

The identity uses *all* Fourier components. Some may be zero; they are simply omitted. There is no search for a nonzero eigenvector and no division by a resolvent. If $\sigma(a) = a$, then a already belongs to the parent fixed field and the entire step can be skipped.

8.2 Principal-root selection

Suppose $R_j \neq 0$, and let $b = R_j^p \in M^K$. Recursively express b by radicals as B. The principal p-th root $v = b^{1/p}$ belongs to M: all roots of $T^p - b$ are $\zeta^k R_j$, and all lie in that field. Therefore there is a unique $\ell \in \{0, \ldots, p-1\}$ such that

$$R_j = \zeta^\ell v. \tag{14}$$

Test this finite list using exact embedded algebraic equality, and return $\zeta_p^\ell B^{1/p}$. Because B and b are the same complex number, their principal powers agree. This is why the selected embedding must be preserved during recursion.

The phase is computed from the compact internal field value b, not from a repeatedly expanded radical output. The selected phase is recorded in the report. Neither a small imaginary part nor a nearest approximate root is accepted as a certificate. In particular, a negative real cube root generally requires a nonzero phase relative to the principal complex cube root.

8.3 The bottom field

At recursion level zero, $a \in C$. Write uniquely

$$a = \sum_{k=0}^{\varphi(m)-1} q_k \zeta_m^k, \qquad q_k \in \mathbb{Q}. \tag{15}$$

Replace ζ_m by $(-1)^{2/m}$. This is already a radical expression, so recursion ends. The Python implementation solves the coordinate system for this cyclotomic basis inside M and checks the full reconstructed vector, not only a square subsystem. Wolfram uses `ToNumberField` with the actual cyclotomic generator; a rational bottom value is returned directly.

8.4 Pseudocode

```
Encode(a, level):
    enforce the node budget; use memoization
    if a = 0: return 0
    if level = 0: return cyclotomic-coordinate expression for a
    (K, N, sigma, p) = chain[level]
    assert a is fixed by N
    if sigma(a) = a: return Encode(a, level - 1)
    compute all R[j] from the p-term orbit of a
    assert R[0] is fixed by K
    terms = [Encode(R[0], level - 1)]
    for j = 1, ..., p-1:
        if R[j] = 0: append 0; continue
        b = R[j]^p
        assert b is fixed by K
        B = Encode(b, level - 1)
        v = principal_pth_root(b), embedded exactly in M
        find ell with zeta_p^ell * v = R[j], exactly
        append zeta_p^ell * principal_pth_root(B)
    return sum(terms)/p
```

9 Correctness and completeness of the construction

Theorem 9.1 (Constructive sufficiency)**.** *Assume the exact field operations, primitive-element computations, factorizations and embedded algebraic comparisons described above are available. If α is solvable by radicals, the unbounded construction returns a radical expression equal to the specified α.*

Proof. The splitting field and cyclotomic compositum are finite, so their exact construction terminates under the stated primitives. The relative group H is solvable by the lemma in Section 6. Thus the finite group procedure reaches the identity and produces (11).

Prove by induction on the recursion level that `Encode(a,level)` is a permitted radical expression equal to a, whenever a lies in the corresponding fixed field. At level zero this is (15). At a positive level, the Fourier lemma puts every recursively needed value in the parent field. The induction hypothesis supplies its radical expression; (14) reconstructs each nonzero resolvent with the correct complex value. The inverse Fourier identity reconstructs a. Zero resolvents and skipped fixed-field steps preserve the same conclusion. Every recursive call reduces the level, so the process is finite. At the top level the fixed subgroup is trivial and the input $\alpha \in M$ is allowed. □

Theorem 9.2 (Decision in the unbounded mathematical model)**.** *For every exact algebraic α over $\mathbb{Q}$, the construction either returns correct radicals or proves that no such radicals exist, provided the stated exact primitives terminate.*

Proof. The solvable case is the previous theorem. Otherwise the relative group is nonsolvable. The normal-series procedure encounters a nontrivial perfect subgroup in finitely many strict subgroup reductions. The cyclotomic-base lemma then implies that the minimal polynomial's Galois group is nonsolvable. The existence criterion rules out radicals for its specified root. □

Remark 9.3 (A theorem is not a blanket software warranty)*.* The reference implementations invoke large CAS primitives, do not implement a formally verified algebraic-number kernel, and impose finite default budgets. A backend exception remains a backend exception even with the budgets removed. The completeness theorem describes the algorithm with terminating exact primitives; it is not a claim that SymPy or a particular Wolfram release successfully executes every such primitive for every input.

9.1 The supplied-model caveat

Python also accepts a separately constructed, validated Galois field model. It checks normality, cyclotomic containment, actual automorphism actions and (10). However, that field can be larger than the target's normal closure. A nonsolvable supplied model *does not* prove the target nonsolvable: even a rational number belongs to nonsolvable extensions. The low-level supplied-model routine therefore raises a backend/model error in this situation, not a target-level `NotSolvable` conclusion. The automatic construction has the stronger minimal-splitting-field provenance needed for that conclusion.

10 Certification details and software architecture

10.1 Keep field values and radical syntax separate

Internal values are canonical algebraic numbers or rational vectors modulo a primitive polynomial. Output values are ordinary radical expressions. Each recursion node maintains both views. Expanding the radical expression is unnecessary for applying automorphisms; operations on the internal value are smaller and make fixed-field checks direct. Memoization keys use exact field coordinates and recursion level.

Wolfram obtains the coordinates of $A(\theta)$ from `AlgebraicNumber`, in ascending power order. The SymPy field representation uses descending coefficient lists for Horner evaluation. Mixing these conventions would silently corrupt automorphism actions. The accompanying implementations make the order explicit.

10.2 Exactness of the chosen embedding in Python

The final Python equality check first computes monic minimal polynomials. Different polynomials imply different values. Equal degree-one polynomials identify the same rational value. For higher degree, equality of minimal polynomials alone only proves conjugacy. The implementation additionally calls `Poly.same_root`, which uses a polynomial root-separation bound and bounded-error evaluation to identify the selected root [6, 7].

This is materially different from a fixed working-precision test. If a separation bound is $\delta > 0$, approximations with certified errors small relative to δ can distinguish conjugates even when their first many decimal digits agree. The regression suite includes

$$1 + \sqrt{2}\,10^{-30} \quad \text{and} \quad 1 - \sqrt{2}\,10^{-30}.$$

Their minimal polynomials are equal; their selected values are not.

The default real PSLQ path of SymPy's `to_number_field` checks an exact defining-polynomial relation but uses a high-precision match to select which conjugate that relation represents. For the package's phase and target embeddings, `embed_exact` instead calls `field_isomorphism` with `fast=False`. That factorization path selects linear factors with the same-root test. The final public check is independent of this coordinate construction. This distinction was found by inspecting the installed SymPy source, not by assuming that every high-level call with exact inputs automatically provides an independent branch certificate.

The package still trusts the CAS's minimal-polynomial and bounded-evaluation implementations. Its `Verified` flag means a checked CAS computation, not an exported proof object that a tiny standalone verifier or Lean kernel can validate. Exporting full isolating-region and field-arithmetic certificates would be a useful further project.

10.3 Failure taxonomy

Status	Meaning
`Success`	The expression passed the radical grammar and selected-root check.
`NotSolvable`	An exact group computation proves the minimal polynomial's group nonsolvable.
`Unknown`	Fast mode found no certified structural conversion. No impossibility conclusion follows.
`ResourceLimit`	A time, degree, recursion or node budget was reached. No impossibility conclusion follows.
`BackendError`	A CAS operation failed, a required representation was unavailable, or an invariant did not verify.
`InvalidInput`	The Wolfram front end rejected the contract; the Python front end instead raises input-validation exceptions.

In particular, `Unknown`, `ResourceLimit`, and `BackendError` are never converted into `NotSolvable`. A finite unsuccessful search cannot certify a negative mathematical statement.

11 Complexity: finite does not mean small

Let $n = \deg f$, $d = [L : \mathbb{Q}]$, $D = [M : \mathbb{Q}]$, and $h = |H|$. Then

$$d \le n!, \qquad D \le d\varphi(\mathrm{rad}(d)), \qquad D = h\varphi(m).$$

Even moderate root degree can therefore lead to a large primitive polynomial. The degree cap is a guard on a mathematical quantity, not a complete memory or bit-complexity bound. Rational coefficient heights can grow drastically while the degree remains unchanged.

There is a useful bound on the *Fourier expression recursion*, considered separately. Ignoring memoization, a level with quotient prime p_i makes at most p_i recursive calls. Hence the tree for one value has at most

$$1 + p_r + p_r p_{r-1} + \cdots + \prod_{i=1}^{r} p_i \le 2h - 1$$

nodes, because every prime is at least two. At most $h - 1$ nontrivial resolvent-root extractions occur in the full tree. This does not bound the printed formula by $O(h)$ characters: bottom coefficients and repeated cyclotomic expressions can be large, and coefficient bit lengths are not bounded by this count.

The reference implementation builds a full D-by-D automorphism multiplication table, performs polynomial substitutions for field actions, and checks fixed-field invariants repeatedly. With naive dense arithmetic, these steps are expensive even before accounting for exact factorization, primitive-element construction and embedding selection. No polynomial-time claim in n, input length, or output length is made.

The Python degree cap is checked after each successive field adjunction; a single oversized adjunction can consume substantial resources before its resulting degree is known. The Wolfram primitive-element call constructs several generators together, so its wall-clock limit is particularly useful.

Python has no solver-internal wall-clock option. The supplied test runner uses subprocess timeouts, and a production service should similarly isolate expensive computations in a worker process.

The implemented fast paths are therefore substantive, not decorative. A Dickson polynomial of degree seven or a reciprocal sextic can be solved without constructing their full splitting fields. The general construction is a fallback and an executable specification for more efficient future backends.

12 Using the supplied implementations

12.1 Wolfram Language

Load the package directly from its extracted folder; a paclet descriptor is also included. Direct loading avoids depending on paclet installation.

```
Get["/path/to/RadicalRoots/Kernel/RadicalRoots.wl"];
r = Root[-1 - #^2 - #^3 + #^4 + #^6 &, 2];
report = RadicalRootReport[r, "Method" -> "Fast"];
report["Status"]
report["Expression"]
RadicalEqualQ[report["Expression"], r]
```

`RadicalRoot[r]` returns the expression on success and a `Failure` object otherwise.

`RadicalRootReport[r]` returns an association with status, method, verification flag and details. The default method is `Automatic`; alternatives are `"Fast"` and `"Galois"`. The public syntax and equality predicates are `RadicalExpressionQ` and `RadicalEqualQ`.

The default budgets are 120 seconds, field degree 64, 10,000 Fourier nodes and fast-recursion depth 30. Explicit extension hints are supported:

```
RadicalRootReport[r,
  "ExtensionHints" -> {Sqrt[2], 3^(1/3)}]
```

The list identifies a compositum, not a list of separately attempted fields. The hint-based factorization must still produce a certified smaller factor that the fast layer can solve. To request the general algorithm without the implemented budgets:

```
RadicalRootReport[r, "Method" -> "Galois",
  "TimeConstraint" -> Infinity,
  "MaxFieldDegree" -> Infinity,
  "MaxNodes" -> Infinity,
  "MaxDepth" -> Infinity]
```

This can be extremely expensive. It does not disable backend limitations, system memory limits or every possible internal CAS recursion limit. Numbers already returned as radicals by initial normalization may bypass the general construction even when that method is requested.

12.2 Python and SymPy

Install the pinned dependency and run the examples from the package root:

```
python -m pip install -r python/requirements.txt
python examples/original_examples.py
python examples/embedded_model.py
python tests/run_tests.py --timeout 40
```

For direct use, put the package's `python` directory on `sys.path` or `PYTHONPATH`:

```
import sympy as s
from radicalroots import to_radicals
x = s.Symbol("x")
r = to_radicals(x**6+x**4-x**3-x**2-1, 1,
                method="fast")
assert r.status == "Success" and r.verified
print(r.expression)
```

The methods are `"auto"`, `"fast"`, and `"galois"`. `None` disables each of `max_field_degree`, `max_nodes`, and `max_depth`. There is no internal time option. The result is a `RadicalResult` dataclass with `status`, `expression`, `method`, `verified`, and `details`.

The standalone automatic model does not require a special Galois-group function for arbitrary degree. It computes its own embedded automorphisms from the splitting field. SymPy's group routine, documented for irreducible rational polynomials through degree six, is only an optional early negative decision accelerator [5]. Thus this limit is not an artificial degree-six restriction of the mathematical backend.

12.3 Supplying a known embedded field

A field prepared elsewhere can avoid expensive automatic discovery:

```
from galois_core import (GaloisModel, radicals_from_model,
                         embed_exact)
K = s.QQ.algebraic_field(s.sqrt(2), s.sqrt(3))
model = GaloisModel.from_field(K, conductor=2)
a = embed_exact(K, s.sqrt(2)-s.sqrt(3))
e, details = radicals_from_model(model, a)
```

This interface requires an actual embedded Galois field containing the specified principal root of unity. The conductor must include every prime dividing the relative group order. An abstract group or a list of approximate roots does not satisfy the contract.

13 Validation record and limitations

The final isolated regression run completed **36 tests: 36 passed, zero failed, and zero timed out**, with a 40-second limit per test. The machine-readable `test-results/python-tests.json` records the actual Python version, SymPy version, platform, subprocess timeout, elapsed time and output for each regression. The tests cover the following distinct issues:

- Every conjugate of the motivating sextic and quintic, plus low-degree, shifted-binomial, degree-seven Dickson and composition identities.

- Full automatic Fourier conversion for quadratic, cubic and a nonabelian degree-eight splitting-field example; supplied real and complex biquadratic models; cyclotomic-base-only descent.
- Nonzero phase corrections, zero resolvents, near-coincident conjugates, strict output syntax, invalid input, invalid field models, resource limits, and exact nonsolvability of an S_5 example.

The finite-group component is independently exercised on C_5, S_3, S_4, the dihedral group of order eight, and the perfect group A_5. The S_5 negative test uses $X^5 - X - 1$ and checks the exact reported order 120. The reducible-input test additionally selects a rational root from a polynomial containing that nonsolvable factor.

The final test-result file is authoritative for pass counts and timings; `README.md` summarizes it. Tests are not a proof of universal program correctness. In particular, the suite does not establish practical automatic splitting-field performance on arbitrary solvable quintics, septics or the large polynomials in the notebook. Early exploratory adjunctions of multiple `CRootOf` generators were slow enough to motivate the low-degree internal-radical optimization. No claim is made to have run the complete notebook or to have benchmarked a competitive high-degree Galois solver.

The native Wolfram kernel was unavailable in the authoring environment. The connected Wolfram service returned HTTP 404 before evaluation on both attempts. Therefore the supplied `.wlt` tests are *not* reported as passed. A static delimiter/comment/string check is useful for packaging, but is not a substitute for executing the package. The Python results do not certify the Wolfram translation.

There are also intentional scope limitations. The solver does not denest or optimize a returned tower globally. Real inputs may produce complex intermediate radicals. It does not provide an all-real-radical guarantee, which is a stronger condition. Input recognition from numerical constants, parameter stratification and proofs about inverse special functions remain outside its contract. Failure reports preserve these distinctions rather than silently returning an unchecked expression.

14 Related work and practical next steps

The theoretical ingredients are classical: solvability of finite groups, Galois correspondence, cyclotomic base change and cyclic-extension resolvents. A recent explicit algorithmic treatment by Song Li also uses Lagrange/Fourier-resolvent ideas [8]. The present package is an independently written reference implementation organized around actual embedded-field actions, a concrete compositum that fixes the needed roots of unity, and explicit principal-branch selection. It does not adopt unverified runtime claims for a different representation or implementation.

Several optimizations can preserve the exact contract. A stronger backend could construct relative fields rather than a large absolute primitive field, use group stabilizers and subfield algorithms to find smaller towers, or compute only the field actions needed for one target. Recognized cyclotomic periods and solvable quintic normal forms could be additional fast paths. Such accelerators should propose exact structures, then pass the same invariant and branch checks; none should replace a failed search with a nonsolvability claim.

A further useful separation is an independently checkable certificate: primitive polynomials and isolating regions, rational matrices for automorphisms, subgroup inclusions, resolvent powers, and branch-isolating certificates. Those data could allow an external verifier to check an answer without repeating field discovery. This release records decisions and verifies them in the CAS, but it does not yet export that full proof format.

The central conclusion is practical as well as theoretical. The original examples do not require blind searches over complicated extension guesses; their structures give short radicals directly. For the remaining solvable cases, a complete constructive route is available. Its main obstacle in a general-purpose implementation is the cost and robustness of exact field computation, not the absence of a mathematical algorithm.

A Package map and reproducibility

Path	Purpose
`Kernel/RadicalRoots.wl`	Wolfram package and both conversion layers.
`Kernel/init.m`, `PacletInfo.wl`	Loader and optional paclet metadata.
`python/radicalroots.py`	Public Python interface, structural proposals and selected-root checks.
`python/galois_core.py`	Exact field models, automorphisms, prime series and Fourier descent.
`python/requirements.txt`	Pinned SymPy dependency.
`tests/test_python.py`	Executable regressions.
`tests/run_tests.py`	Isolated runner with actual JSON results.
`tests/RadicalRoots.wlt`	Native Wolfram regression specification.
`tests/run_wolfram.wls`	Native test runner.
`examples/`	Original examples and supplied-model demonstration.
`test-results/`	Actual Python results and explicit Wolfram execution status.
`source-review/`	Notebook inspection, transcription, provenance and hashes.
`docs/article.tex`, `docs/article.pdf`	This article.

Compile the article from its directory with two runs of `pdflatex -interaction=nonstopmode -halt-on-error article.tex`. The bibliography is embedded; no BibTeX invocation is required. Runtime measurements include test-process startup and should not be read as isolated solver benchmarks. The new implementation is MIT-licensed; the original notebook-derived material retains its separate attribution.

References

[1] Vladimir Reshetnikov, *Why is ToRadicals not able to handle all cases? Is there a workaround?*, Mathematica Stack Exchange, question 34011. `https://mathematica.stackexchange.com/questions/34011/why-is-toradicals-not-able-to-handle-all-cases-is-there-a-workaround`.

[2] Wolfram Research, *ToRadicals*, Wolfram Language documentation. `https://reference.wolfram.com/language/ref/ToRadicals.html`.

[3] Wolfram Research, *ToNumberField* and *RootReduce*, Wolfram Language documentation. `https://reference.wolfram.com/language/ref/ToNumberField.html`; `https://reference.wolfram.com/language/ref/RootReduce.html`.

[4] J. S. Milne, *Fields and Galois Theory*, version 5.10, 2022, especially Chapter 5 and Theorem 5.34. `https://www.jmilne.org/math/CourseNotes/FT.pdf`.

[5] The SymPy development team, *Number Fields*, SymPy 1.14.0 documentation. `https://docs.sympy.org/latest/modules/polys/numberfields.html`.

[6] The SymPy development team, *Polynomials Manipulation Module Reference*, entry `Poly.same_root`, SymPy 1.14.0 documentation. `https://docs.sympy.org/latest/modules/polys/reference.html`.

[7] The SymPy development team, SymPy 1.14.0 source code, `sympy/polys/numberfields/subfield.py` and `sympy/polys/polytools.py`. The installed source was inspected for `field_isomorphism_pslq`, `field_isomorphism_factor`, and `Poly.same_root`. `https://github.com/sympy/sympy/tree/1.14.0/sympy/polys`.

[8] Song Li, *An Algorithm for Solving Solvable Polynomial Equations of Arbitrary Degree by Radicals*, arXiv:2203.11602v2, 2022. `https://arxiv.org/abs/2203.11602`.

Online references consulted 8 September 2026. The supplied notebook is identified separately by its archive and notebook SHA-256 hashes in `source-review/README.md`.